Appendix C: R Code

Nadav Kempinski

December 18, 2025

Introduction

R code for analyzing sample data across Californian source waters. Citations for all packages utilized are at the end of the document.

Downloading & Formatting Sample Data

```
library(googlesheets4) # to import concentrations data from google sheets, a
    shared spreadsheet repo
library(janitor)
library(tidyverse)
```

```
# download raw data from Google Sheets
data_file = read_sheet("your/spreadsheet/url/here") # Don't use na="-" as it
    messes up the detection_method column

# Format & normalize data
data_clean = data_file |> # cleaned, omitted lat & lon only
  clean_names() |> # normalize column names
  mutate(
    keep = coalesce(keep, "Not a Duplicate"),
    detection_method = na_if(detection_method, "-"),
    sample_unit = tolower(sample_unit), # normalize units
    sample_date = as.Date(sample_date),
    detection_limit_unit = tolower(detection_limit_unit),
    longitude = as.numeric(longitude), # set data values for lat & lon to
        help with plotting
    latitude = as.numeric(latitude),
```

```
    detection_method = coalesce(detection_method, "Unknown"),
    sample_concentration = if_else(
      sample_unit == "ug/l",
      sample_concentration * 1000,
      sample_concentration),
    sample_unit = "ng/l",
    detection_limit = if_else(
      detection_limit_unit == "ug/l",
      detection_limit * 1000,
      detection_limit),
    detection_limit_unit = "ng/l",
    analyte = as.factor(analyte), water_body = as.factor(water_body)) # for
        making factors of the analytes

# list of non-duplicated data (before QA/QC fix)
data_zeros_no_qa = data_clean |> filter(keep !="remove") # includes
    non-detects

# review: samples with QA/QC issues
flag_terms <- data_clean |> filter(qa_issues == 1, !is.na(qa_term)) |>
    select(qa_term) |> unique() |> pull(qa_term)
added_terms = c("received warm", "has been exceeded", "time exceeded", "warm
    when received")
to_flag = unique(c(keepers, flag_terms))

# apply dictionary to all samples & flag matches
regex_terms <- paste0("(", paste(to_flag, collapse = "|"), ")")
data_zeros_flagged = data_zeros_no_qa |>
  mutate(qa_flag = str_detect(notes, regex(regex_terms, ignore_case=TRUE)),
         qa_flag_term = str_extract(notes, regex(regex_terms, ignore_case =
             TRUE))
  )

# count totals
count(data_zeros_flagged |> filter(qa_flag)) # flagged for QA/QC issues
count(data_zeros_flagged |> filter(!qa_flag)) # not flagged for QA/QC issues

# check non-flagged for false-negatives
set.seed(42)
sample_qa = data_zeros_flagged |> filter(!qa_flag) |> slice_sample(n=100) |>
    select(sample_id, notes)
```

```
cat(
  paste0(
    sprintf("%d/%d) ID=", seq_len(nrow(sample_qa)), nrow(sample_qa)),
    sample_qa$sample_id, "\n",
    sample_qa$notes
  ),
  sep = "\n\n---\n\n"
)

n_unflagged = sum(!data_zeros_flagged$qa_flag)
upper_lim = binom.test(0,100)$conf.int[2]
hi95 = ceiling(n_unflagged * upper_lim)

# Keep those not flagged for data analysis
data_zeros = data_zeros_flagged |> filter(!qa_flag)

data_detects = data_zeros |> # detections only
  filter(sample_concentration_normalized > 0) # remove all 0 dots so we can
      just see the detected results
```

Mapping Samples

```
library(rnaturalearth) # map-making
library(sf) # for map-making
library(tidyverse)
library(tigris)
options(tigris_use_cache = TRUE) # to save sf layers instead of
    re-downloading
```

```
## Create a map of CA
ca_state_shp = ne_states(country="United States of America", returnclass =
    "sf") |> filter(name=="California") # get US shapefile & filter for CA
# get base layer coordinate system so we can project all layers over
ca_state_shp_crs = st_crs(ca_state_shp)
ca_counties_shp = tigris::counties(state="CA") |>
  st_transform(ca_state_shp_crs) # get CA counties
ca_rivers_shp = ne_download(scale="large", type = "rivers_lake_centerlines",
    category="physical", returnclass="sf") |> st_intersection(ca_state_shp)
    |> # get rivers
  st_transform(ca_state_shp_crs) # get CA rivers & lakes
```

```r
ca_cities_shp = ne_download(scale = "large", type = "populated_places",
    category = "cultural", returnclass = "sf") |> filter(ADM1NAME ==
    "California") |> # get cities
  st_transform(ca_state_shp_crs) # get CA population centers
ca_lakes_shp = ne_download(scale = "large", type = "lakes", category =
    "physical", returnclass = "sf") |> # get lakes
  st_make_valid() |> # because of weird cropping issues
  st_intersection(ca_state_shp) |> # crop to CA
  st_transform(ca_state_shp_crs) # get CA lakes

# create date range to title the samples map, include in base map
data_range_year = paste0(year(min(data_zeros$sample_date)), "-",
    year(max(data_zeros$sample_date)))

ca_base_map = ggplot() +
  geom_sf(data = ca_state_shp, fill = "white", color = "black") +
  geom_sf(data = ca_counties_shp, color = "gray60", fill = NA, size = 0.2) +
  geom_sf(data = ca_rivers_shp, color = "lightblue") +
  geom_sf(data = ca_lakes_shp, color = "lightblue", fill = "lightblue") +
  theme_void() +
  theme(
    panel.background = element_rect(fill = "white", color = NA),
    plot.background = element_rect(fill = "white", color = NA)
  ) +
  labs(title=str_wrap(paste0("California Map of Tobacco-Related Analyte
      Samples (", data_range_year, ")"), width=60))
ca_base_map

## FIGURE: Map of CA with all samples. Must state grays in caption
ca_samples_map <- ca_base_map +
  # Plot non-detects in gray
  geom_point(data = data_zeros |> filter(sample_concentration_normalized==0),
             aes(x = longitude, y = latitude, shape = analyte),
             fill = "gray90", color = "black", size = 2, alpha = 0.5) +

  # Plot detects with log-scaled fill
  geom_point(data = data_detects,
             aes(x = longitude, y = latitude, shape = analyte, fill =
                 sample_concentration_normalized),
             color = "black", size = 2, alpha = 0.8) +

  scale_fill_gradientn(
```

```
    name = "Concentration (ng/L)",
    colors = c("skyblue", "blue", "red", "black"),
    values = scales::rescale(log10(c(1, 10, 100, 1000))),
    trans = "log",
    breaks = c(1, 10, 100, 1000),
    labels = c("1", "10", "100", "1000")
  ) +

  scale_shape_manual(
    name = "Analyte",
    values = c("Cotinine" = 21, "Nicotine" = 24)
  )

ca_samples_map # will remind you that Masoner et al. (2019) data (n=8) is
    missing, as it has no geolocation data!
```

Downloading Data from Esri Map Services

```
library(sf)
library(ggplot2)
#remotes::install_github("yonghah/esri2sf"), # install.packages("pbapply")
library(esri2sf)
```

```
saveSHP = function(url, outputName = "") {

  attempt = try(features <- esri2df(url))
  if(inherits(attempt, "try-error")) {
    print(paste0("Error using esri2df: ", attempt))
    attempt2 = try(features <- esri2sf(url))
    if(inherits(attempt2, "try-error")) {
      print(paste0("Error using esri2sf: ", attempt2))
      attempt3 = try(st_read(paste0(url,
    "/query?where=1=1&outFields=*&f=json")))
      if(inherits(attempt3, "try-error")) {
        print(paste0("Error using st_read (stopping here): ", attempt3))
      } else {
        features = attempt3
      }
    }
  }
```

```
  if (outputName != "") {
    outputFile = paste0(outputName, ".shp")
    sf::st_write(features, outputFile, append = FALSE)
  }

  return(features)
}

gis_layer = saveSHP(url_gis_layer)
```

Classifying samples by groundwater basin

```
library(sf)
library(tidyverse)
library(kableExtra)
```

```
# California Groundwater Basin Boundary Shapefile Data can be downloaded
    from: https://water.ca.gov/programs/groundwater-management/bulletin-118.
    Note that I've saved it as "basins_sf". Direct link (which needs to be
    unzipped):
    https://gis.data.cnra.ca.gov/api/download/v1/items/49807a1fbc584631bdf88d9ca71dd083/shape
basins_sf = st_read("Downloads/i08_B118_CA_GroundwaterBasins") |>
    st_transform(4326)

# take detect & non-detect data & convert into Simple Feature (sf)
data_sf = data_zeros |>
  drop_na(latitude, longitude)
sample_data <- st_as_sf(data_sf, coords=c('longitude', "latitude"), crs=4326)

## join to find major basin number for each sample
sf::sf_use_s2(FALSE) # due to duplicate vertex issue
samples_with_basins <- st_join(sample_data, basins_sf[, c("Basin_Numb")],
    left = TRUE) |>
  mutate(basin_major = str_extract(Basin_Numb, "^[^-]+")) |>
  filter(water_body == "Groundwater")
sf::sf_use_s2(TRUE) # make sure the option is re-toggled for later projects

# Summarize sample count by basin
samples_with_basins |>
```

```
  group_by(basin_major) |>
  summarize(count=n()) |>
  kable() |>
  kable_styling()
```

Data analysis for paper inclusion

```
library(tidyverse)
library(kableExtra)
library(patchwork)
library(ggridges)
```

```
# --- Executive Summary
data_zeros |>
  mutate(detected = ifelse(sample_concentration_normalized > 0, TRUE, FALSE))
      |>
  group_by(analyte, detected) |>
  summarize(count=n())

### Number of Duplicates
count(data_clean |> filter(keep == "remove")) # 3728

### Paper stats
# compilation: # of data points (including duplicates)
count(data_clean) # 7277

# compilation: # of non-dups
count(data_zeros) # 3,549

# compilation: non-detects - 2,730
count(data_zeros |> filter(sample_concentration_normalized<=0))

# compilation: final dataset - 819
count(data_detects)

# --- Contaminant Detection

# % detections per analyte-body
data_zeros %>%
  group_by(analyte, water_body) %>%
```

```
  summarize(
    total = n(),
    detected = sum(sample_concentration_normalized > 0, na.rm = TRUE),
    detection_rate = detected / total
  )

# --- TPW Chem Detection Methods

# compilation: LC-MS/MS count - 1477
count(data_zeros |> filter(phase_type == "LC-MS/MS"))

# compilation: GC-MS count - 183
count(data_zeros |> filter(phase_type == "GC-MS"))

# compilation: unknown count - 1889
count(data_zeros |> filter(phase_type == "Unknown"))

# compilation: no detection limit - 37
count(data_zeros |> filter(is.na(detection_limit)))

# compilation: sw8270C use - 31
count(data_zeros |> filter(detection_method == "SW8270C"))

### TABLE: Detection Methods Limits
detection_ranges_table <- data_zeros %>% # de-duped w/ non-detects
  filter(!is.na(detection_limit), !is.na(detection_limit_unit)) |> # remove
      where detection limit isn't defined, n=48. These two clauses overlap
  group_by(analyte, phase_type) %>%
  summarize(
    min_limit = min(detection_limit),
    max_limit = max(detection_limit),
    Methods = str_c(sort(unique(detection_method)), collapse = "; "),
    `Detection Range` = paste0(min(detection_limit), " - ",
        max(detection_limit)),
    Sources = str_c(sort(unique(data_source)), collapse = ", "),
    Count=n(),
    Detections = sum(sample_concentration_normalized > 0),
    .groups = "drop"
  ) %>%
  rename(Analyte = analyte,
         `Phase Type` = phase_type) |>
  select(Analyte, `Phase Type`, `Detection Range`, Methods, Sources, Count,
      Detections) |>
```

```
  arrange(Analyte)
detection_ranges_table |> kable() |> kable_styling()

## TABLE: Range by analyte-body
detection_ranges <- data_detects %>%
  group_by(analyte, water_body) %>%
  summarize(
    min_conc = min(sample_concentration_normalized, na.rm = TRUE),
    max_conc = max(sample_concentration_normalized, na.rm = TRUE),
    n_detects = n(),
    .groups = "drop"
  )
detection_ranges

# --- Discussion

# % cotinine of total - 75.0%
count(data_zeros |> filter(analyte == "Cotinine")) / count(data_zeros)

# mean of each analyte-water
data_detects |>
  group_by(analyte, water_body) |>
  summarize(mean = mean(sample_concentration_normalized), .groups="drop") |>
  kable() |> kable_styling()

# count: missing lat&lon (8, all from Masoner et al. (2019))
data_clean |>
  group_by(is.na(latitude),is.na(longitude)) |>
  summarize(count=n())

# find duplicates (4,177, 2,077 to keep)
data_clean |>
  group_by(keep != "") |>
  summarize(count=n())

# get 0's (2,730 of non-duplicates)
data_zeros |>
  group_by(sample_concentration == 0) |>
  summarize(count = n(),
            .groups = "drop")


# TABLE: get counts by analyte & water body
```

```r
table_sources_zeros <- data_zeros %>%
  group_by(analyte, water_body) %>%
  summarize(count = n(), .groups = "drop") %>%
  pivot_wider(names_from = water_body, values_from = count, values_fill = 0)
      |>
  mutate(`Total` = rowSums(across(-analyte)))
table_sources_zeros |> kable() |> kable_styling()

table_sources_zeros2 = table_sources_zeros |>
  summarize(across(where(is.numeric), sum)) |>
  mutate(analyte = "TOTAL") |>
  relocate(analyte)
table_sources_zeros3 = bind_rows(table_sources_zeros, table_sources_zeros2)
table_sources_zeros3 |> kable() |> kable_styling()


table_sources_detects <- data_detects %>%
  group_by(analyte, water_body) %>%
  summarize(count = n(), .groups = "drop") %>%
  pivot_wider(names_from = water_body, values_from = count, values_fill = 0)
      |>
  mutate(`Total` = rowSums(across(-analyte)))

table_sources_detects2 = table_sources_detects |>
  summarize(across(where(is.numeric), sum)) |>
  mutate(analyte = "TOTAL") |>
  relocate(analyte)
table_sources_detects3 = bind_rows(table_sources_detects,
    table_sources_detects2)
table_sources_detects3 |> kable() |> kable_styling()

library(ggplot2)
library(dplyr)
library(patchwork)

# Function to annotate sample size (n=)
sample_sizes <- data_detects %>%
  group_by(analyte, water_body) %>%
  summarize(n = n(), .groups = "drop")

# FIGURE: boxplots of concentrations by analyte-water body
library(ggplot2)
library(dplyr)
```

```r
# Filter valid values
# Get sample sizes per group
sample_sizes <- data_detects %>%
  group_by(analyte, water_body) %>%
  summarize(n = n(), .groups = "drop")

# Color palette by analyte
analyte_colors <- c(
  "Cotinine" = "#F8766D",  # reddish
  "Nicotine" = "#00BFC4"   # teal
)

# Function to generate plot per analyte
make_boxplot <- function(analyte_name) {
  # Filter and rename estuary to brackish

  label_df <- filter(sample_sizes, analyte == analyte_name) %>%
    mutate(
      water_body = ifelse(water_body == "Estuary", "Brackish Water",
          as.character(water_body))
    )

  plot_df <- filter(data_detects, analyte == analyte_name) %>%
    mutate(
      water_body = ifelse(water_body == "Estuary", "Brackish Water",
          as.character(water_body))
    ) %>%
    left_join(label_df, by = c("analyte", "water_body")) %>%
    mutate(
      water_body_labeled = paste0(water_body, " (n=", n, ")"),
      method = ifelse(n < 5, "strip", "box")
    )

  label_df <- label_df %>%
    mutate(water_body_labeled = paste0(water_body, " (n=", n, ")"))
  # Build plot
  ggplot(plot_df, aes(x = water_body_labeled, y =
      sample_concentration_normalized)) +
    geom_boxplot(
      data = subset(plot_df, method == "box"),
      fill = analyte_colors[[analyte_name]],
      alpha = 0.6,
```

```
      outlier.shape = NA
    ) +
    geom_jitter(
      width = 0.15,
      size = 1,
      alpha = 0.3,
      color = "black"
    )+
    scale_y_log10(
      breaks = c(1, 10, 100, 1000),
      labels = c("1", "10", "100", "1000")
    ) +
    labs(
      x = "Water Sampled",
      y = "Concentration (ng/L, log10 scale)",
      title = paste(analyte_name)
    ) +
    theme_minimal(base_size = 12) +
    theme(
      axis.text.x = element_text(angle = 45, hjust = 1),
      plot.title = element_text(hjust = 0.5)
    )
}

# Create one plot per analyte
plots <- lapply(unique(data_detects$analyte), make_boxplot)

# Combine plots with a centered main title
boxplots = wrap_plots(plots) +
  plot_annotation(
    title = "Detected Analyte Concentrations by Waters Sampled (ng/L)",
    theme = theme(plot.title = element_text(hjust = 0.5, size = 14, face =
        "bold"))
  )


## FIGURE: Ridgeplot of analyte x water sampled (good for highlighting
    distribution)
ridgeplots = ggplot(data_detects, aes(x = sample_concentration_normalized, y
    = water_body, fill = analyte)) +
  ggridges::geom_density_ridges(
    scale = 1.2,
    alpha = 0.6,
```

```
    rel_min_height = 0.01,
    color = "black"
  ) +
  scale_x_log10(
    breaks = c(1, 10, 100, 1000),
    labels = c("1", "10", "100", "1000")
  ) +
  labs(
    x = "Concentration (ng/L, log10 scale)",
    y = "Sampled Water Source",
    title = "Analyte Concentration Distribution",
    fill = "Analyte"
  ) +
  scale_fill_manual(values = c("Cotinine" = "#F8766D", "Nicotine" =
      "#00BFC4")) +
  theme_minimal(base_size = 12) +
  theme(
    strip.text = element_text(face = "bold"),
    legend.position = "right",
  )

analyte_concentrations_fig = boxplots + ridgeplots
analyte_concentrations_fig

## STATS: compare detection method/phase type by analyte/water source
data_binary <- data_zeros %>%
  filter(phase_type %in% c("LC-MS/MS", "GC-MS")) %>%
  mutate(
    detected = sample_concentration_normalized > 0
  )

detection_summary <- data_binary %>%
  group_by(analyte, water_body, phase_type) %>%
  summarise(
    n_total = n(),
    n_detect = sum(detected, na.rm = TRUE),
    detection_rate = round(100 * n_detect / n_total, 1),
    .groups = "drop"
  )
detection_summary

test_detection_method <- function(df) {
  tbl <- table(df$phase_type, df$detected)
```

```
  if (all(dim(tbl) == c(2, 2))) {
    test <- fisher.test(tbl)
    return(tibble(
      p_value = test$p.value,
      method = "Fisher",
      total_n = sum(tbl),
      table = list(tbl)
    ))
  } else {
    return(tibble(p_value = NA, method = NA, total_n = sum(tbl), table =
        list(tbl)))
  }
}

# Run test per group
test_results <- data_binary %>%
  group_by(analyte, water_body) %>%
  group_modify(~ test_detection_method(.x)) %>%
  ungroup()

summary_combined <- detection_summary %>%
  pivot_wider(
    names_from = phase_type,
    values_from = c(n_total, n_detect, detection_rate)
  ) %>%
  left_join(test_results, by = c("analyte", "water_body")) |>
  mutate(significant = ifelse(!is.na(p_value) & p_value < 0.05, "Yes", "No"))
      |>
  filter(`n_total_LC-MS/MS` > 0, `n_total_GC-MS` > 0)

summary_combined |> kable() |> kable_styling()


## STAT: differences in mean concentration between analytes
water_bodies <- unique(data_detects$water_body)

wilcox_test_results <- lapply(water_bodies, function(water) {
  df_subset <- data_detects %>%
    filter(water_body == water)

  if (length(unique(df_subset$analyte)) == 2) {
    result <- wilcox.test(sample_concentration_normalized ~ analyte, data =
    df_subset)
```

```
    print(result$p.value)
    tibble(
      water_body = water,
      p_value = result$p.value,
      mean_cotinine =
          mean(df_subset$sample_concentration_normalized[df_subset$analyte ==
          "Cotinine"]),
      mean_nicotine =
          mean(df_subset$sample_concentration_normalized[df_subset$analyte ==
          "Nicotine"]),
      n_cotinine = sum(df_subset$analyte == "Cotinine"),
      n_nicotine = sum(df_subset$analyte == "Nicotine")
    )
  }
}) %>% bind_rows()
wilcox_test_results |> kable() |> kable_styling()
# GW: p=1.39e-14. SW: p=1.52e-5

## STAT: Difference in water body mean for same analyte
# For Cotinine only
cotinine_data <- data_detects %>%
  filter(analyte == "Cotinine", !is.na(sample_concentration_normalized),
      !is.na(water_body))

pairwise.wilcox.test(
  x = cotinine_data$sample_concentration_normalized,
  g = cotinine_data$water_body
)

nicotine_data <- data_detects %>%
  filter(analyte == "Nicotine", !is.na(sample_concentration_normalized),
      !is.na(water_body))

pairwise.wilcox.test( #2e-7
  x = nicotine_data$sample_concentration_normalized,
  g = nicotine_data$water_body,
  p.adjust.method = "holm"
)

## count: # samples per body
data_zeros |>
  group_by(water_body, analyte) |>
```

```
  summarize(n=n(), .groups = "drop") |>
  kable() |> kable_styling()


## check: by source, who is receiving sample data for each analyte?
data_zeros |>
  # filter(str_detect(data_source, "\\(20")) |>
  group_by(data_source, analyte) |>
  summarize(count=n(), .groups="drop") |>
  group_by(data_source) |>
  summarize(analytes=n(), .groups="drop") |>
  arrange(desc(analytes)) |>
  kable() |> kable_styling()
```

Citations

Analyses were conducted using the R Statistical language (version 4.4.1; R Core Team, 2024) on Windows 11 x64 (build 26100), using the packages googlesheets4 (version 1.1.1; Bryan J, 2023), janitor (version 2.2.0; Firke S, 2023), lubridate (version 1.9.3; Grolemund G, Wickham H, 2011), esri2sf (version 0.1.1; Hwang Y et al., 2025), report (version 0.6.1; Makowski D et al., 2023), rnaturalearth (version 1.0.1; Massicotte P, South A, 2023), tibble (version 3.2.1; Müller K, Wickham H, 2023), sf (version 1.0.19; Pebesma E, Bivand R, 2023), patchwork (version 1.3.0; Pedersen T, 2024), tigris (version 2.1; Walker K, 2024), ggplot2 (version 3.5.1; Wickham H, 2016), forcats (version 1.0.0; Wickham H, 2023), stringr (version 1.5.1; Wickham H, 2023), tidyverse (version 2.0.0; Wickham H et al., 2019), dplyr (version 1.1.4; Wickham H et al., 2023), purrr (version 1.0.2; Wickham H, Henry L, 2023), readr (version 2.1.5; Wickham H et al., 2024), tidyr (version 1.3.1; Wickham H et al., 2024), ggridges (version 0.5.6; Wilke C, 2024) and kableExtra (version 1.4.0; Zhu H, 2024).

References

  - Bryan J (2023). _googlesheets4: Access Google Sheets using the Sheets API V4_. R package version 1.1.1, <https://CRAN.R-project.org/package=googlesheets4>.
  - Firke S (2023). _janitor: Simple Tools for Examining and Cleaning Dirty Data_. R package version 2.2.0, <https://CRAN.R-project.org/package=janitor>.
  - Grolemund G, Wickham H (2011). "Dates and Times Made Easy with lubridate." _Journal of Statistical Software_, *40*(3), 1-25. <https://www.jstatsoft.org/v40/i03/>.

- Hwang Y, Kevin C, Jacob P (2025). _esri2sf: Create Simple Features from ArcGIS Server REST API_. R package version 0.1.1, commit f47eda534b22de0bc903def20ac45397295333dc, <https://github.com/yonghah/esri2sf>.
- Makowski D, Lüdecke D, Patil I, Thériault R, Ben-Shachar M, Wiernik B (2023). "Automated Results Reporting as a Practical Tool to Improve Reproducibility and Methodological Best Practices Adoption." _CRAN_. <https://easystats.github.io/report/>.
- Massicotte P, South A (2023). _rnaturalearth: World Map Data from Natural Earth_. R package version 1.0.1, <https://CRAN.R-project.org/package=rnaturalearth>.
- Müller K, Wickham H (2023). _tibble: Simple Data Frames_. R package version 3.2.1, <https://CRAN.R-project.org/package=tibble>.
- Pebesma E, Bivand R (2023). _Spatial Data Science: With applications in R_. Chapman and Hall/CRC. doi:10.1201/9780429459016 <https://doi.org/10.1201/9780429459016>, <https://r-spatial.org/book/>. Pebesma E (2018). "Simple Features for R: Standardized Support for Spatial Vector Data." _The R Journal_, *10*(1), 439-446. doi:10.32614/RJ-2018-009 <https://doi.org/10.32614/RJ-2018-009>, <https://doi.org/10.32614/RJ-2018-009>.
- Pedersen T (2024). _patchwork: The Composer of Plots_. R package version 1.3.0, <https://CRAN.R-project.org/package=patchwork>.
- R Core Team (2024). _R: A Language and Environment for Statistical Computing_. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>.
- Walker K (2024). _tigris: Load Census TIGER/Line Shapefiles_. R package version 2.1, <https://CRAN.R-project.org/package=tigris>.
- Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_. Springer-Verlag New York. ISBN 978-3-319-24277-4, <https://ggplot2.tidyverse.org>.
- Wickham H (2023). _forcats: Tools for Working with Categorical Variables (Factors)_. R package version 1.0.0, <https://CRAN.R-project.org/package=forcats>.
- Wickham H (2023). _stringr: Simple, Consistent Wrappers for Common String Operations_. R package version 1.5.1, <https://CRAN.R-project.org/package=stringr>.
- Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R, Grolemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E, Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi K, Vaughan D, Wilke C, Woo K, Yutani H (2019). "Welcome to the tidyverse." _Journal of Open Source Software_, *4*(43), 1686. doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.
- Wickham H, François R, Henry L, Müller K, Vaughan D (2023). _dplyr: A Grammar of Data Manipulation_. R package version 1.1.4, <https://CRAN.R-project.org/package=dplyr>.
- Wickham H, Henry L (2023). _purrr: Functional Programming Tools_. R package

version 1.0.2, <https://CRAN.R-project.org/package=purrr>.
  - Wickham H, Hester J, Bryan J (2024). _readr: Read Rectangular Text Data_. R package version 2.1.5, <https://CRAN.R-project.org/package=readr>.
  - Wickham H, Vaughan D, Girlich M (2024). _tidyr: Tidy Messy Data_. R package version 1.3.1, <https://CRAN.R-project.org/package=tidyr>.
  - Wilke C (2024). _ggridges: Ridgeline Plots in 'ggplot2'_. R package version 0.5.6, <https://CRAN.R-project.org/package=ggridges>.
  - Zhu H (2024). _kableExtra: Construct Complex Table with 'kable' and Pipe Syntax_. R package version 1.4.0, <https://CRAN.R-project.org/package=kableExtra>.